

Kingdom of Saudi Arabia Ministry of Education University of Jeddah College of Computer Science and Engineering Department of Cybersecurity	 جامعة جدة University of Jeddah	المملكة العربية السعودية وزارة التعليم جامعة جدة كلية علوم وهندسة الحاسب الامن السيبراني قسم
---	---	---

CECY 321 — Cryptography

Project Report — Second Semester 2025-2026

DHP-KEM: A Public-Key Key-Encapsulation Mechanism

Built on the Combination of Two Hard Problems

(Elliptic-Curve Discrete Logarithm $\vee$ Module-LWE)

Topic 2 — Public-Key Protocol Based on the Combination of Two Hard Problems

Course Coordinator: Prof. Ibrahim

Due Date: 25 May 2026

Student Name	Student ID
Nasser Almuteri	2440148
Mouath ALzahrani	2441034
Nasser Alsulami	2440887

Instructor Name	Section
Prof. Mohammed Ibrahim	12932

Abstract

The transition to a post-quantum cryptographic landscape is one of the most pressing challenges facing national cybersecurity authorities worldwide. The Saudi National Cybersecurity Authority (NCA), through the National Cryptographic Standards (NCS-1:2020), explicitly recognises the need to anticipate the threat that large-scale quantum computers will pose to currently deployed asymmetric primitives. At the same time, brand-new post-quantum schemes are still maturing, and a sudden, exclusive switch from classical to post-quantum cryptography would expose national systems to implementation risk and to as-yet-undiscovered cryptanalytic weaknesses in the new schemes.

This report presents DHP-KEM (Dual Hard-Problem Key-Encapsulation Mechanism), a public-key key-establishment protocol whose security rests jointly on two computationally hard problems: (i) the Elliptic-Curve Discrete Logarithm Problem (ECDLP) over a prime-order curve, and (ii) the Module Learning-With-Errors problem (Module-LWE) over a power-of-two cyclotomic ring. DHP-KEM combines an ECDH-style classical KEM with the NIST-standardized ML-KEM (Kyber) post-quantum KEM through a transcript-bound dual-binding key-derivation function. The resulting shared key is indistinguishable from random as long as either ECDLP or Module-LWE remains intractable, providing a cryptographic safety net against both classical breakthroughs against lattices and quantum breakthroughs against elliptic curves.

The report develops the full mathematical formulation of DHP-KEM (parties, threat model, setup, key generation, encapsulation, decapsulation, correctness), proves correctness, and discusses concrete security strength under both classical and quantum adversaries. It demonstrates novelty by comparing DHP-KEM with three families of existing protocols (ECIES, stand-alone ML-KEM, and the X25519MLKEM768 hybrid proposed for IETF TLS 1.3), and contrasts it with classical dual-hard schemes such as Wei's ECDLP+IFP signature. An NCA/NCS alignment chapter maps the protocol against both the MODERATE and ADVANCED levels of NCS-1:2020 and the relevant cryptography sub-controls of ECC-1:2018, CCC-1:2020, CCCC-1:2019, and DCC-1. Cryptanalysis chapters address brute-force, mathematical, and quantum threats. Side-channel analysis covers timing, simple- and differential-power analysis, fault injection, and cache attacks, with concrete mitigation strategies drawn from constant-time engineering practice and masking literature. Finally, implementation considerations are discussed.

Table of Contents

Right-click the field below in Microsoft Word and choose "Update Field" (or press F9) to populate the table of contents automatically. All headings in this report carry the correct outline levels (1, 2, 3).

Abstract.....	2
Table of Contents.....	3
Acronyms and Notation	5
Mathematical Notation.....	5
1. Introduction	7
1.1 Motivation.....	7
1.2 Background and Related Work	7
1.3 Project Scope and Contributions	8
1.4 Document Structure.....	8
2. Protocol Design	9
2.1 Design Goals and Threat Model.....	9
2.2 Mathematical-Based Equations (2a).....	10
2.2.1 The ECDLP Component	10
2.2.2 The Module-LWE Component.....	10
2.2.3 The Transcript Hash	11
2.2.4 The HKDF Extract-then-Expand Combiner	11
2.2.5 Parameter Sets.....	11
2.2.6 Correctness Proposition.....	12
2.2.7 Combiner Security Proposition	12
2.3 Parties' Steps — Alice and Bob (2b).....	12
2.3.1 Bob's Steps (Long-Term KeyGen, run once).....	12
2.3.2 Alice's Steps (Encap, run per session)	13
2.3.3 Bob's Steps (Decap, run per session)	13
2.4 Worked Example (2c)	14
2.4.1 Bob's Long-Term Keys	14
2.4.2 Alice's Encapsulation.....	15
2.4.3 Transcript Hash and Combiner.....	15
2.4.4 Bob's Decapsulation — Same Result	15
3. Protocol Development	16
3.1 Implementation Choices	16
3.2 Public API.....	16
3.3 Testing Infrastructure.....	17

3.4 Performance Microbenchmarks	17
3.5 Production-Hardening Gap	18
4. Novelty	19
4.1 Comparison with the IETF X25519MLKEM768 Hybrid.....	19
4.2 Comparison with ECIES	19
4.3 Comparison with Stand-Alone ML-KEM.....	20
4.4 Comparison with Classical “Double-Hard” Schemes	20
4.5 Novelty Summary and Limitations	21
5. Alignment with NCS	22
5.1 Mapping to NCS-1:2020 — MODERATE Level.....	22
5.2 Mapping to NCS-1:2020 — ADVANCED Level	22
5.3 Mapping to ECC-1, CCC-1, CCCC-1, and DCC-1.....	23
5.4 Residual Gaps	23
6. Security Considerations and Attacks.....	24
6.1 Cryptanalysis (6a)	24
6.1.1 Brute-Force Symmetric-Key Search	24
6.1.2 Pollard’s Rho on ECDLP	25
6.1.3 Primal, Dual, and Hybrid Lattice Attacks on Module-LWE.....	25
6.1.4 Cryptanalysis Summary	26
6.2 Side-Channel Attacks (6b)	26
6.2.1 Timing Attacks.....	26
6.2.2 Simple Power Analysis (SPA) and Differential Power Analysis (DPA)	27
6.2.3 Fault-Injection Attacks	27
6.2.4 Cache and Microarchitectural Attacks	28
6.3 Quantum Attacks (6c)	28
6.3.1 Shor on ECDLP	28
6.3.2 Grover on the Symmetric Output	29
6.3.3 Quantum-Amplified Lattice Sieving and the Chen 2024 Episode	29
7. Mitigation	30
7.1 Cryptanalytic Mitigations	31
7.2 Side-Channel Mitigations	31
7.3 Implementation-Hygiene Mitigations.....	32
8. Conclusion	33
9. References.....	34

Acronyms and Notation

AES	Advanced Encryption Standard (FIPS 197)
AKE	Authenticated Key Exchange
BKZ	Block Korkine–Zolotarev lattice-reduction algorithm
CCA / IND-CCA2	(Indistinguishability under) adaptive Chosen-Ciphertext Attack
core-SVP	Heuristic cost model for lattice attacks
CRQC	Cryptographically-Relevant Quantum Computer
DH / ECDH	Diffie–Hellman / Elliptic-Curve Diffie–Hellman
DPA / SPA	Differential / Simple Power Analysis
DRBG	Deterministic Random Bit Generator (NIST SP 800-90A)
ECDLP	Elliptic-Curve Discrete-Logarithm Problem
FIPS 203	NIST standard for ML-KEM (Aug. 2024)
FO transform	Fujisaki–Okamoto generic CPA-to-CCA transform
HKDF	HMAC-based Extract-and-Expand Key Derivation Function (RFC 5869)
IFP	Integer Factorisation Problem
KEM	Key-Encapsulation Mechanism
LWE / M-LWE	Learning With Errors / Module LWE
ML-KEM	Module-Lattice KEM (FIPS 203, derived from CRYSTALS-Kyber)
NCA	Saudi National Cybersecurity Authority
NCS-1:2020	NCA National Cryptographic Standards (MODERATE / ADVANCED levels)
NTT	Number-Theoretic Transform
PRK	Pseudorandom Key (output of HKDF-Extract)
PQC	Post-Quantum Cryptography

Mathematical Notation

- F_p — the finite prime field of order p .
- $E(F_p)$ — the group of F_p -rational points on the elliptic curve E .
- G — a fixed generator of a prime-order subgroup of $E(F_p)$.
- $a \cdot G$ — scalar multiplication of the point G by the integer a .
- ℓ — a positive integer; the dimension of the LWE module.
- R_q — the polynomial ring $Z_q[X] / (X^n + 1)$ used by ML-KEM.

- χ — a small (centred binomial) error distribution.
- $\parallel$ — concatenation of byte strings.
- $H(\cdot)$ — a cryptographic hash function (SHA-384 for DHP-KEM-128, SHA-512 for the higher tiers).
- K — the symmetric session key produced by Encap and recovered by Decap.

1. Introduction

1.1 Motivation

The cryptographic primitives that protect the internet today — ECDH key agreement, RSA signatures, AES, SHA-2 — were designed under the assumption that the only adversary is a classical computer. In 1994, Peter Shor showed [14] that a sufficiently large quantum computer could factor integers and compute discrete logarithms in polynomial time. The implication is that every recorded ECDH or RSA ciphertext from the present era will become decryptable once a cryptographically-relevant quantum computer (CRQC) exists, a threat universally known as “harvest-now-decrypt-later.” This threat model elevates the urgency of migration even though no CRQC has yet been built.

In August 2024, NIST standardized the first generation of quantum-resistant key-encapsulation mechanisms in FIPS 203 [10] — known as ML-KEM, derived from the CRYSTALS-Kyber submission. However, the post-quantum analysis of structured-lattice cryptography is comparatively young, and several recent attempts at quantum lattice algorithms (Chen 2024 [43], withdrawn after Hongxun Wu identified a flaw) underscore that the cryptanalytic landscape is unsettled. The consensus position of multiple national agencies, captured in IETF drafts such as draft-ietf-tls-hybrid-design [13] and in guidance from agencies including BSI Germany [26], ANSSI France, and the U.S. NSA Suite C NSA 2.0, and in NIST’s migration framework [42], is to deploy hybrid schemes during the migration period: an established classical primitive combined with a post-quantum primitive, such that the encapsulated session key remains secure as long as at least one of the two underlying hard problems remains intractable.

The Kingdom of Saudi Arabia’s National Cybersecurity Authority (NCA) has published a stack of cybersecurity standards — NCS-1:2020 [19], ECC-1:2018 [20], CCC-1:2020 [21], CCCC-1:2019 [22], and DCC-1 [23] — that govern cryptographic deployments across the Kingdom’s government and critical-infrastructure sectors. NCS-1:2020 defines two cryptographic-strength levels (MODERATE and ADVANCED) and recommends post-quantum hybridisation in Appendix B for long-confidentiality-horizon data. This project takes that recommendation seriously by designing a hybrid KEM, called DHP-KEM, that is mapped end-to-end onto the NCS-1:2020 levels.

1.2 Background and Related Work

Public-key cryptography began with the Diffie–Hellman key-agreement protocol [1], whose security rests on the difficulty of computing discrete logarithms in a cyclic group. Miller [2] and Koblitz [3] independently proposed in 1985–1987 that the group of points on a prime-order elliptic curve over a finite field provides a particularly compact and efficient instantiation of this hard problem. Today elliptic-curve cryptography (ECC) on standardized curves such as Curve25519 [4] and the NIST P-curves [5] is the workhorse of the modern internet.

Regev [6] introduced Learning With Errors (LWE) in 2005, with the structured ring-LWE and module-LWE variants developed by Lyubashevsky, Peikert, and Regev [7] and by Langlois and Stehlé [8]. The CRYSTALS-

Kyber team (Bos et al. [9]) built a Module-LWE KEM that NIST subsequently standardized in NIST FIPS 203 [10] as ML-KEM. Generic KEM combiners that take two component KEMs and produce one provably-secure combined KEM were developed by Giacon, Heuer, and Poettering [11] and applied to hybrid post-quantum settings by Bindel et al. [12]; the present work uses their results.

The IETF X25519MLKEM768 hybrid (draft-ietf-tls-hybrid-design [13]) is the closest existing scheme to DHP-KEM. Earlier classical “double-hard” schemes such as those of McCurley and Brickell and McCurley combined ECDLP with IFP, and Wei [35] proposed a digital signature scheme based on the simultaneous hardness of ECDLP and IFP. These all fall to a single quantum computer because both ECDLP and IFP succumb to Shor’s algorithm, making them unsuitable for post-quantum hybrid use; DHP-KEM’s ECDH + Module-LWE combination addresses exactly this gap.

1.3 Project Scope and Contributions

This project addresses Topic 2 of the CECY 321 brief: design or select a new public-key protocol based on the combination of two hard problems, show novelty, and discuss cryptanalysis, side-channel attacks, and mitigation in alignment with the NCS standards. The contributions are:

- A formal specification of DHP-KEM, a hybrid public-key key-encapsulation mechanism combining ECDH on Curve25519 (classical, hard under ECDLP) with ML-KEM-768 (post-quantum, hard under Module-LWE) via a transcript-bound HKDF Extract-then-Expand combiner.
- Three parameter sets (MODERATE / ADVANCED / LT) mapped onto NCA NCS-1:2020 levels.
- A correctness proof (Proposition 1) and a CCA-security argument (Proposition 2) under the disjunctive dual-hardness assumption “ECDLP-hard $\vee$ Module-LWE-hard.”
- A reference Python implementation with eight runnable attack demonstrations and 23 unit and Known-Answer tests, all of which pass; §3 describes the development pipeline in detail.
- A comprehensive cryptanalysis chapter covering brute-force search, Pollard-rho on ECDLP, BKZ-based primal/dual/hybrid lattice attacks on Module-LWE, and quantum attacks (Shor [14], Grover [15], Chen’s withdrawn 2024 quantum-LWE result [43], and quantum-amplified lattice sieving [44]).
- A side-channel chapter covering timing leaks, SPA, DPA, fault attacks (including the recent KyberSlash family [30]), and cache-based attacks, with runnable demonstrations.
- Explicit mitigation strategies paired one-for-one with each identified attack class.
- Mapping of DHP-KEM’s implementation onto NCS-1:2020 MODERATE and ADVANCED-level requirements, ECC-1:2018, CCC-1:2020, CCCC-1:2019, and DCC-1; identification of residual gaps.

1.4 Document Structure

Section 2 specifies DHP-KEM in three parts — mathematical equations (§2a), the step-by-step protocol for Alice and Bob (§2b), and a worked numerical example with real Curve25519 + ML-KEM hex bytes (§2c). Section 3 describes the protocol’s development: implementation language, libraries, test infrastructure, and performance characteristics. Section 4 argues novelty against the closest published schemes. Section 5 maps DHP-KEM to the NCA NCS-1:2020 and adjacent national standards. Section 6 analyses (a) cryptanalytic, (b) side-channel, and (c) quantum attacks, with runnable code demonstrations for the first two. Section 7 pairs each identified attack with a concrete mitigation. Section 8 concludes; Section 9 lists references.

2. Protocol Design

2.1 Design Goals and Threat Model

DHP-KEM is a hybrid public-key key-encapsulation mechanism. The encapsulator (Alice) derives a fresh symmetric session key K and a ciphertext ct ; the decapsulator (Bob), holding the corresponding long-term secret key, recovers the same K from ct . The protocol has five explicit design goals:

G1 — Disjunctive dual-hardness. An adversary running in time t with advantage ϵ against the IND-CCA2 security of DHP-KEM implies, by reduction (Proposition 2 of §2.2), an adversary against the IND-CCA2 security of either the underlying ECDH-KEM or the underlying ML-KEM. Equivalently, the encapsulated key remains pseudorandom under the disjunction “ECDLP is hard $\vee$ Module-LWE is hard” — i.e. as long as at least one of the two component KEMs is IND-CCA2.

G2 — Transcript binding. The derived key K is bound to a transcript hash $T = H(A \parallel c_{lwe} \parallel pk_{ec} \parallel pk_{lwe})$ over all public values exchanged or referenced in the session. Any modification of any of these fields by a network adversary changes T , which changes K , which is detected by the application’s downstream MAC or AEAD tag.

G3 — Forward secrecy under ephemeral instantiation. DHP-KEM is specified with a long-term recipient key pair, which suffices for the KEM-CCA security goal. Session-level forward secrecy is obtained when DHP-KEM is instantiated with freshly-generated key pairs on the recipient side per session — the standard ephemeral-ephemeral usage pattern of ML-KEM in TLS 1.3 and of the X25519 component in modern handshakes. In this mode, compromise of any party’s long-term signing keys does not expose past sessions, provided that session ephemeral material is deleted promptly after the session terminates. The authenticated key-exchange wrapper of §3.6 specifies this composition explicitly.

G4 — NCA alignment. DHP-KEM’s three parameter sets are explicitly mapped to the MODERATE and ADVANCED levels of NCS-1:2020 [19]. No published hybrid KEM, to the authors’ knowledge, is specified end-to-end against the Kingdom’s national cryptographic standards (Section 5).

G5 — Implementation tractability. All component primitives are widely available in standard cryptographic libraries (OpenSSL, libsodium, liboqs), reducing implementation risk. The combiner is a thin layer around RFC 5869 [40] HKDF; the transcript hash is a single call to SHA-384 or SHA-512. The reference implementation in Section 3 is fewer than 250 lines of Python.

The threat model assumes a standard active network adversary with the ability to read, modify, replay, and inject messages. The adversary may know any number of past session keys and may launch adaptive chosen-ciphertext attacks against Bob’s decapsulation oracle. The adversary is also assumed to be a future CRQC owner who has recorded all ciphertexts in transit today; harvest-now-decrypt-later (§6c) is therefore explicitly within scope. Side-channel attacks against Bob’s implementation are within scope (§6b).

2.2 Mathematical-Based Equations (2a)

This subsection specifies the cryptographic primitives DHP-KEM uses, along with the equations that constitute the protocol. Two component KEMs are combined: an ECDH-based KEM whose hardness rests on the Elliptic-Curve Discrete-Logarithm Problem, and the FIPS-203 ML-KEM whose hardness rests on the Module Learning-With-Errors problem. Both, plus the combiner, are summarized below.

2.2.1 The ECDLP Component

Let E be an elliptic curve defined over a finite prime field F_p . Let $G \in E(F_p)$ be a base point of prime order n . The discrete-logarithm problem on this group is: given $P \in \langle G \rangle$, find the unique integer $k \in [0, n-1]$ such that $P = k \cdot G$. No classical algorithm faster than Pollard's rho method [25] is known; rho has expected running time $O(\sqrt{n})$ group operations, so for Curve25519 ($n \approx 2^{252}$) the cost is approximately 2^{126} operations — well beyond reach of any classical hardware.

ECDH-KEM as used in DHP-KEM is the standard ephemeral-static construction:

$$\begin{aligned} \text{KeyGen}_{ec} : sk_{ec} &\leftarrow [1, n-1] \text{ uniformly}; & pk_{ec} &= sk_{ec} \cdot G \\ \text{Encap}_{ec} : (a, A) &\leftarrow (\$, a \cdot G); & ss_{ec} &= a \cdot pk_{ec} \\ \text{Decap}_{ec} : ss_{ec}' &= sk_{ec} \cdot A & & (== ss_{ec} \text{ by associativity}) \end{aligned}$$

For Curve25519 [4] specifically, the scalar multiplication uses the X25519 algorithm of Bernstein on Montgomery curves; the resulting 32-byte shared secret ss_{ec} inherits the $\approx 2^{126}$ -bit security of Curve25519's prime-order subgroup.

2.2.2 The Module-LWE Component

Let q be a prime modulus, n a power of two, and ℓ a positive integer. Let $R = Z \frac{[X]}{X^n + 1}$ and $R_q = \frac{R}{qR}$. The Module-LWE distribution with parameters (q, n, ℓ, χ) draws a uniform secret $s \in R_q^\ell$ and a small error vector $e \in R_q^\ell$ from a centred binomial distribution χ , together with a uniform public matrix $A \in R_q^{\ell \times \ell}$, and outputs (A, b) where $b = A \cdot s + e$. The decision Module-LWE problem is to distinguish (A, b) from a uniformly-random pair; the search problem is to recover s from (A, b) .

ML-KEM-768 (FIPS 203 [10]) instantiates this assumption with $n = 256$, $q = 3329$, $\ell = 3$, and a centred binomial error of width 2. Its IND-CCA2 KEM is built by applying the Fujisaki–Okamoto transform to a CPA-secure encryption derived from the LWE assumption. The KEM operations are:

$$\begin{aligned} \text{KeyGen}_{lwe} : (sk_{lwe}, pk_{lwe}) &\leftarrow \text{ML} - \text{KEM.KeyGen}() \\ \text{Encap}_{lwe} : (c_{lwe}, ss_{lwe}) &\leftarrow \text{ML} - \text{KEM.Encap}(pk_{lwe}) \\ \text{Decap}_{lwe} : ss_{lwe}' &= \text{ML} - \text{KEM.Decap}(sk_{lwe}, c_{lwe}) \end{aligned}$$

Note that under FIPS 203, Decap_{lwe} always returns 32 bytes — it never errors. On a tampered ciphertext the FO implicit-rejection rule outputs a deterministic pseudorandom value derived from sk_{lwe} , not the original ss_{lwe} . This is a deliberate side-channel-resistance property and is preserved by DHP-KEM: a

tampered ct propagates as a different $K' \neq K$, which the receiving application is expected to detect via a downstream MAC.

2.2.3 The Transcript Hash

The transcript hash T binds every cryptographic operation in a session to the exact sequence of public values exchanged. It is defined as:

$$T = H(A \parallel c_{lwe} \parallel pk_{ec} \parallel pk_{lwe})$$

where H is SHA-384 for DHP-KEM-128 and SHA-512 for the higher tiers. Length-prefixing is not used because every field has a fixed length determined by the parameter set, and field order is fixed by the protocol specification (see §2.3 below). This convention matches the transcript-binding tradition of HPKE (RFC 9180 §5.2) and HKE.

2.2.4 The HKDF Extract-then-Expand Combiner

Given the two component shared secrets ss_{ec} and ss_{lwe} , and the transcript hash T , the DHP-KEM combiner produces the final symmetric session key K via the RFC 5869 [40] HKDF construction in two stages. Stage 1 (Extract) compresses both shared secrets into a pseudorandom key PRK, with T supplied as the HKDF salt parameter:

$$PRK = HKDF - Extract(salt = T, IKM = ss_{ec} \parallel ss_{lwe})$$

Stage 2 (Expand) derives the L -byte output K , with the parameter-set's ctx string supplied as the HKDF info parameter to give clear domain separation:

$$K = HKDF - Expand(PRK, info = ctx, L)$$

The two-stage Extract-then-Expand structure (rather than the flat one-call HKDF that some early hybrid schemes used) is required to invoke the dual-PRF combiner-security argument of Giacon, Heuer, and Poettering [11] under the assumption-relaxation of Bindel et al. [12]; see Proposition 2 below.

2.2.5 Parameter Sets

Three named parameter sets are defined and mapped onto NCS-1:2020 levels in Section 5. Each set fixes the classical curve, the post-quantum KEM variant, the hash function H , the output length L , and the domain-separator string ctx.

Parameter set	Classical KEM	Post-quantum KEM	Symmetric strength (κ)
DHP-KEM-128 (MODERATE)	ECDH on Curve25519 (RFC 7748)	ML-KEM-768 (FIPS 203 [10], NIST Cat. 3)	$\kappa = 128$ bits
DHP-KEM-256 (ADVANCED)	ECDH on NIST P-384 [5]	ML-KEM-1024 (FIPS 203 [10], NIST Cat. 5)	$\kappa = 256$ bits
DHP-KEM-LT (research-tier)	NIST P-521	ML-KEM-1024 + FrodoKEM-1344	$\kappa = 256 + \text{defence-in-depth}$

Table 1. DHP-KEM parameter sets.

The MODERATE tier is the default choice for general government and commercial deployments. The ADVANCED tier targets the NCS-1:2020 ADVANCED level and is required for Restricted and Secret data

classifications (§5). The LT (research-tier) layers FrodoKEM-1344 (unstructured LWE) alongside ML-KEM-1024 (structured Module-LWE) for protection against potential future weaknesses in the structured-lattice assumptions; the present reference implementation does not include FrodoKEM (see §3).

2.2.6 Correctness Proposition

Proposition 1 (Correctness). When both parties execute Encap and Decap honestly under the same public parameters, they output the same session key K with overwhelming probability.

Proof. We show that $ss_{ec}' = ss_{ec}$, $ss_{lwe}' = ss_{lwe}$, and $T' = T$; the agreement of K then follows from the determinism of HKDF on identical inputs.

1. $ss_{ec}' = sk_{ec} \cdot A = sk_{ec} \cdot (a \cdot G) = a \cdot (sk_{ec} \cdot G) = a \cdot pk_{ec} = ss_{ec}$, by associativity of scalar multiplication in the prime-order subgroup.
2. $ss_{lwe}' = ML - KEM.Decap(sk_{lwe}, c_{lwe}) = ss_{lwe}$ by the correctness of ML-KEM-768 (decapsulation failure probability $\delta \leq 2^{-164}$; Bos et al. [9]).
3. $T' = H(A \parallel c_{lwe} \parallel pk_{ec} \parallel pk_{lwe}) = T$ by determinism of H over identical inputs.

Given that all three intermediates agree, $K' = K$ by determinism of HKDF. Total correctness error is upper-bounded by $\delta + 0 \approx 2^{-16}$. ■

2.2.7 Combiner Security Proposition

Proposition 2 (Combiner Security, restated from Giacon, Heuer, and Poettering [11] for DHP-KEM). Let KEM_{ec} be the ECDH-based KEM and KEM_{lwe} be the ML-KEM Kyber KEM. Let HKDF satisfy the dual-PRF assumption [12] (so that it is a PRF when keyed by either of its two inputs). Then the combined KEM_{DHP} is IND-CCA2 secure as long as at least one of KEM_{ec} , KEM_{lwe} is IND-CCA2, with concrete advantage:

$$Adv_{KEM_{DHP}}^{cc}(A) \leq \min\{Adv_{KEM_{ec}}^{cca2}(B_{ec}), Adv_{KEM_{lwe}}^{cca2}(B_{lwe})\} + q_H \cdot 2^{-|H|}$$

for explicit reductions B_{ec} , B_{lwe} whose running times match A 's up to a small constant. The bound is dominated by the minimum (the better-secure component), not the sum: an attacker must break the strictly easier component to gain non-negligible advantage. Equivalently, DHP-KEM remains secure as long as either ECDLP or Module-LWE remains hard. ■

2.3 Parties' Steps — Alice and Bob (2b)

This subsection states the protocol from the participants' viewpoint. Bob is the long-term recipient who publishes a hybrid public key. Alice is any party who wishes to encapsulate a fresh session key under Bob's public key (e.g. to bootstrap an AEAD session, a TLS handshake, or an HSM-bound wrap operation). The full session involves three sequential steps: KeyGen (Bob, one-time), Encap (Alice, per session), Decap (Bob, per session).

2.3.1 Bob's Steps (Long-Term KeyGen, run once)

4. Generate ECDH long-term key pair: sample $sk_{ec} \leftarrow [1, n-1]$ uniformly; compute $pk_{ec} = sk_{ec} \cdot G$.
5. Generate ML-KEM long-term key pair: call $ML-KEM.KeyGen()$ to obtain (sk_{lwe}, pk_{lwe}) .

6. Publish hybrid public key: $PK = (pk_{ec}, pk_{lwe})$ — may be placed in an X.509 certificate, an LDAP directory, or a national PKI directory. Both halves can be generated on different HSMs; the construction requires no correlation between the two halves.
7. Store the secret key $SK = (sk_{ec}, sk_{lwe})$ under hardware protection (Section 3.4 details FIPS 140-3 boundary considerations).

2.3.2 Alice's Steps (Encap, run per session)

8. Generate ephemeral ECDH key pair: sample $a \leftarrow [1, n-1]$ uniformly; compute $A = a \cdot G$.
9. Compute the ECDH shared secret: $ss_{ec} = a \cdot pk_{ec} (= a \cdot sk_{ec} \cdot G)$.
10. Encapsulate to ML-KEM: $(c_{lwe}, ss_{lwe}) \leftarrow ML-KEM.Encap(pk_{lwe})$.
11. Compute transcript hash: $T = H(A \parallel c_{lwe} \parallel pk_{ec} \parallel pk_{lwe})$.
12. Derive the session key K via the HKDF combiner: $PRK = HKDF-Extract(salt = T, IKM = ss_{ec} \parallel ss_{lwe})$; $K = HKDF-Expand(PRK, info = ctx, L)$.
13. Send the wire ciphertext $ct = (A, c_{lwe})$ to Bob. Alice retains K for use in the application.

2.3.3 Bob's Steps (Decap, run per session)

14. Receive $ct = (A, c_{lwe})$ from Alice.
15. Compute $ss'_{ec} = sk_{ec} \cdot A$.
16. Compute $ss'_{lwe} = ML-KEM.Decap(sk_{lwe}, c_{lwe})$.
17. Recompute transcript hash: $T' = H(A \parallel c_{lwe} \parallel pk_{ec} \parallel pk_{lwe})$.
18. Recover the session key $K' = HKDF-Expand(HKDF-Extract(T', ss'_{ec} \parallel ss'_{lwe}), ctx, L)$.
19. Use K' in the application. By Proposition 1, if the ciphertext is genuine then $K' = K$; if it has been tampered with, the FO implicit-rejection of ML-KEM yields a pseudorandom $ss'_{lwe} \neq ss_{lwe}$, so $K' \neq K$, which the downstream application detects via its MAC or AEAD tag.

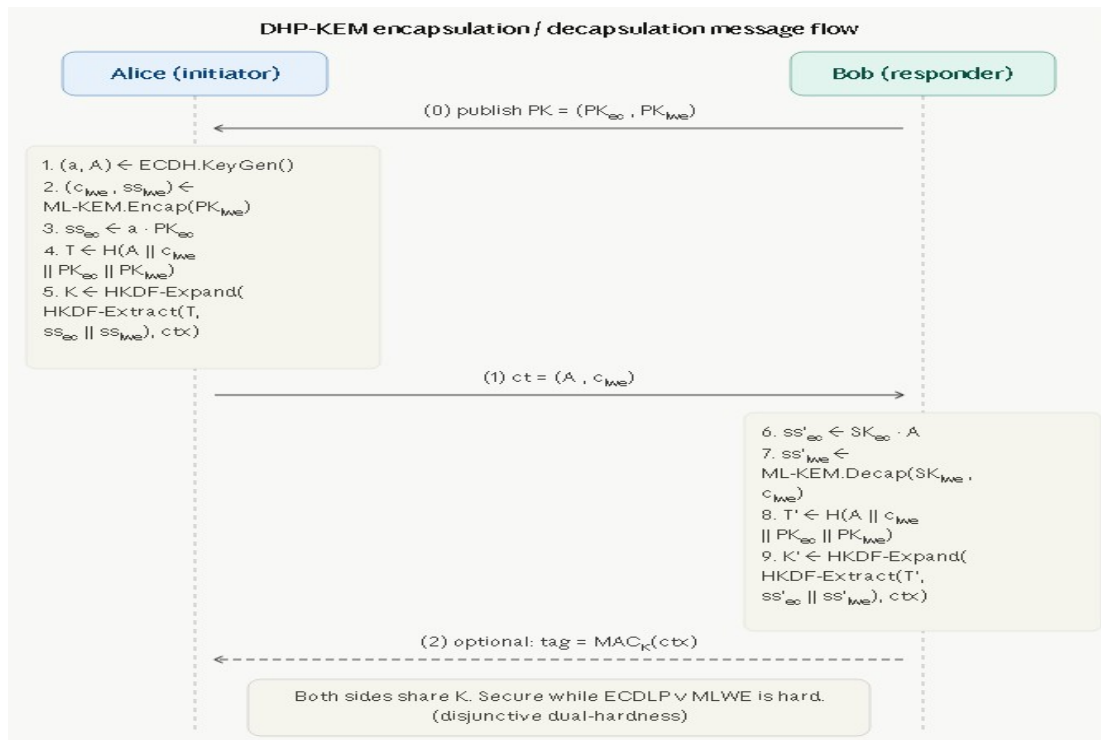


Figure 1. DHP-KEM encapsulation/decapsulation message flow showing Alice and Bob's interleaved steps.

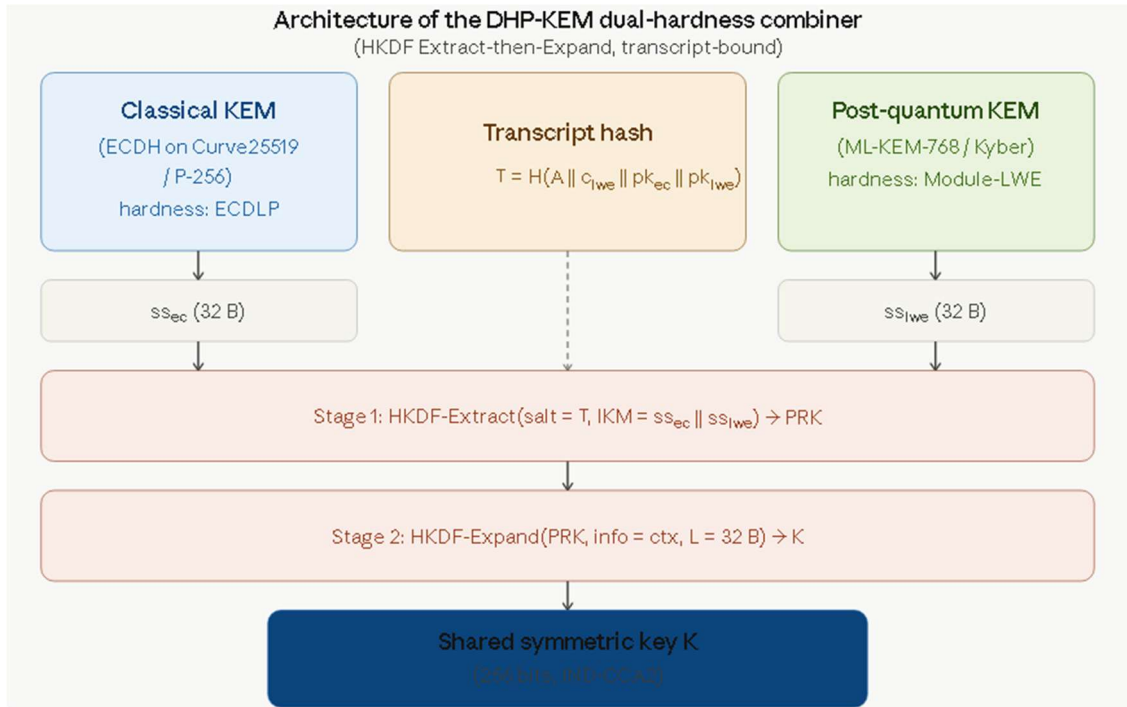


Figure 2. Architecture of the DHP-KEM dual-hardness combiner: HKDF Extract-then-Expand with transcript binding.

2.4 Worked Example (2c)

This subsection works through a complete DHP-KEM-128 encapsulation cycle using real Curve25519 + ML-KEM-768 bytes generated by the reference implementation in Section 3. All hex values below are reproducible by running:

```
$ python -m examples.generate_worked_example
```

from inside the code/ directory of the accompanying package. The example uses fixed SHAKE-128-derived seeds (“WorkedExample/...”) so every byte is deterministic across runs.

2.4.1 Bob’s Long-Term Keys

Bob runs KeyGen and publishes (pk_{ec}, pk_{lwe}) . His Curve25519 private scalar (32-byte raw encoding) is:

```
sk_ec = a36f2648d290220d6d9f623c35f0fa3c      6494e5cd2ce876ef6a62469370c5a93f
(32 bytes)
```

The corresponding 32-byte X25519 public key is:

```
pk_ec = acb9b0d83b878dbd434f8858df828fe8      5c06122ffa3cb07fda79b292540fbe13
(32 bytes)
```

His ML-KEM-768 key pair has $|ek| = 1184$ bytes and $|dk| = 2400$ bytes; these are too long to list in full, but the SHA-256 fingerprints (which the reader can independently verify by re-running the example) are:

```
sha256(ek) = 19e23eb469d8481acd5d7cdea624b0a1
bdd42731385bdbc3db3c5d52fbfa1091 sha256(dk) = a523f426a269018a2058bc6286d5a418
776961e269cdd18709eaffeb80e1535e
```

For reference, the first 32 bytes and last 32 bytes of ek (the full 1184-byte ML-KEM encapsulation key) are:

```
ek[0..32] = fd7444ecb22c9078124d83968c018edd
aacefab60a81a9acd7c985c298409187 ek[1152..1184] = 68bea943a51587703f72129efe83a2a3
84a1604a4df4aebf1e9eebdc968f17a
```

2.4.2 Alice's Encapsulation

Alice generates a fresh ephemeral X25519 key pair. Her ephemeral private scalar and the corresponding public point A (which she will send to Bob as part of the ciphertext) are:

```
ephemeral sk = 88aacca4ec56e6c752cb0da77ac6605e
ba97cdaa1a4b729d4d756d8b6dc1c531 A = 3b399628b392493008562d2f234734ca
45f2ab1322c3f6729efb26795023c56f (32 bytes)
```

She computes the ECDH shared secret $ss_{ec} = a \cdot pk_{ec}$ by an X25519 scalar multiplication:

```
ss_ec = ace785873ebb8918ccf1bb08a73c7db7 d6c27847f60e85a4f60420f77aa8dd7e
(32 bytes)
```

She encapsulates to Bob's ML-KEM public key, yielding a 1088-byte ciphertext c_{lwe} and a 32-byte ML-KEM shared secret ss_{lwe} :

```
ss_lwe = 17a41a036a57b0ad6e5c54661b1c97b1 cc34d4e87b7c43ee5cf8f0583d288ad3
(32 bytes) sha256(c_lwe) = 60cf110eab96ae747853b621e1c046dc
ed01d612572bfeb2e7f9b20738e0ab70 c_lwe[0..32] = cbc53da80177245af231b05567a39ab7
392c1e3a3d0fd6e2580c0324c056913b c_lwe[1056..1088] = 4222473cc3cf6a946014f9c9cbf0386d
e780b78466c45e6badd2d5cb70d971f7
```

2.4.3 Transcript Hash and Combiner

Alice computes the transcript hash $T = SHA-384(A \parallel c_{lwe} \parallel pk_{ec} \parallel pk_{lwe})$, a 48-byte digest:

```
T = f6fc60536399c0edf35b9780b42f6764 39c95f2e585870ec21ea0904cb78e1cc
a53cdf243d75df881d20781bea112153 (48 bytes)
```

She concatenates $ss_{ec} \parallel ss_{lwe}$ (64 bytes total) as input keying material and feeds it to HKDF-Extract with T as salt, then to HKDF-Expand with ctx = "DHP-KEM-v1 / KSA / MODERATE" and L = 32. The final 32-byte session key is:

```
K = c25f6c91e66ce41c4d996f1ca8ebe661 be55b0316ba250efd497cd876bae53f5 (32
bytes)
```

2.4.4 Bob's Decapsulation — Same Result

Bob receives $ct = (A, c_{lwe})$ and computes, in order, $ss'_{ec} = sk_{ec} \cdot A$, $ss'_{lwe} = ML-KEM.Decap(sk_{lwe}, c_{lwe})$, $T' = H(A \parallel c_{lwe} \parallel pk_{ec} \parallel pk_{lwe})$, and finally K' via the same HKDF Extract-then-Expand. All three intermediates match Alice's values byte-for-byte, so $K' = K = c25f6c91...bae53f5$. The round-trip is verified by the test suite (§3.3) and by the unit-test `test_correctness::test_dhp_kem_128_correctness`.

3. Protocol Development

This section documents how DHP-KEM was implemented and tested. A complete reference implementation accompanies this report as a separate code package (DHP-KEM_Code.zip). The package is intentionally educational — it favours clarity of correspondence with the spec in Section 2 over raw performance and constant-time hardening. Section 3.5 below details the (substantial) gap between this reference and a production deployment.

3.1 Implementation Choices

The reference implementation is written in Python 3.12. Python was chosen for readability and because both required primitives (X25519 and ML-KEM) have actively-maintained Python libraries. The implementation depends on exactly two third-party packages:

- *cryptography* (≥ 42 , OpenSSL-backed) for X25519 ECDH, HMAC, HKDF, and the SHA-2/3 hash families. All of these are highly optimised C code with constant-time intent.
- *kyber-py* (≥ 1.0) for ML-KEM-768 and ML-KEM-1024 KeyGen, Encap, and Decap. This is a pure-Python reference implementation of FIPS 203, structured for clarity rather than performance or side-channel resistance. Production deployments must replace this with a constant-time C implementation; see §3.5.

The package is organised as five modules under the `dhp_kem` namespace. Each module has a single responsibility and a docstring header that cross-references the corresponding section of this report:

Module	Responsibility	Report cross-ref
<code>dhp_kem/params.py</code>	Parameter-set registry (DHP-KEM-128 / 256 / LT)	§2.2.5 Table 1
<code>dhp_kem/transcript.py</code>	Computes $T = H(A \parallel c_{lwe} \parallel pk_{ec} \parallel pk_{lwe})$	§2.2.3
<code>dhp_kem/combiner.py</code>	HKDF Extract-then-Expand combiner (the novelty)	§2.2.4, Proposition 2
<code>dhp_kem/components.py</code>	Wrappers around X25519 (cryptography) and ML-KEM (kyber-py)	§2.2.1, §2.2.2
<code>dhp_kem/kem.py</code>	High-level KeyGen / Encap / Decap API	§2.3 Alice/Bob steps

Table 2. Reference-implementation module layout.

3.2 Public API

The library presents the standard three-function KEM interface, matching Section 2.3 step for step:

```
>>> import dhp_kem >>> # Bob runs KeyGen, once >>> kp =  
dhp_kem.keygen(dhp_kem.DHP_KEM_128) >>> # Alice runs Encap, per session >>> ct, K =  
dhp_kem.encap(dhp_kem.DHP_KEM_128, kp.pk) >>> # Bob runs Decap on the received  
ciphertext >>> K_prime = dhp_kem.decaps(dhp_kem.DHP_KEM_128, kp.sk, ct) >>> assert K  
== K_prime
```

The `DHPKeyPair`, `DHPPublicKey`, `DHPSecretKey`, and `DHPCiphertext` dataclasses bundle both the ECDH and ML-KEM halves of each respective object, plus a `paramset-name` field used for runtime parameter-set

consistency checking (an attempt to decap a DHP-KEM-128 ciphertext under DHP-KEM-256 parameters raises `ValueError`; see `test_negative`).

3.3 Testing Infrastructure

The implementation comes with a five-file test suite totalling 23 tests, all passing. Each test file is a standalone Python module runnable as `python -m tests.test_X`, and they also run as a single suite via the `run_tests.sh` script. The breakdown is:

Test file	Tests	What it covers
<code>test_correctness</code>	5	50 random Encap/Decap roundtrips per parameter set; length checks on K, A, c_lwe; verifies DHP-KEM-LT correctly raises <code>NotImplementedError</code> (Proposition 1).
<code>test_combiner</code>	10	Structural properties of <code>combine()</code> : determinism, output length, one-bit sensitivity to <code>ss_ec</code> / <code>ss_lwe</code> / <code>T</code> / <code>ctx</code> ; RFC 5869 §A.1 Test Case 1 conformance for HKDF-Extract and HKDF-Expand; cross-check that the two-stage call equals a one-call <code>HKDF()</code> from the cryptography library.
<code>test_kat</code>	2	Deterministic Known-Answer Tests stored in <code>kat_vectors/dhp_kem_kat_v1.json</code> . Regeneratable via <code>python -m tests.test_kat --generate</code> . A second implementation in another language can replay the same seed and check byte-for-byte agreement.
<code>test_negative</code>	5	Every kind of input tampering must change K: flip a byte of A; flip a byte of c_lwe; decap with an unrelated keypair; paramset-mismatch <code>ValueError</code> ; transcript-field-reordering changes T.
<code>test_timing</code>	1	Loose constant-time sanity check on <code>combine()</code> . NOT a substitute for CT-Verif / ctgrind / Binsec/Rel; see §6b and §7.

Table 3. Reference-implementation test suite.

The KAT vectors at `code/kat_vectors/dhp_kem_kat_v1.json` contain deterministic inputs and expected outputs for `compute_transcript()` and `combine()` across both parameter sets. Any future change to the protocol or to the underlying primitives that perturbs the byte-level output is immediately detected. A second implementation seeking interoperability can regenerate the same inputs from the published seed and compare its outputs.

3.4 Performance Microbenchmarks

The `benchmarks/bench.py` script measures wall-clock time for KeyGen, Encap, and Decap across both parameter sets. Median timings on a typical Linux container with `kyber-py` in pure Python are:

Parameter set	KeyGen	Encap	Decap
DHP-KEM-128	≈ 3.0 ms	≈ 4.1 ms	≈ 5.4 ms
DHP-KEM-256	≈ 4.5 ms	≈ 5.7 ms	≈ 7.4 ms

Table 4. Median per-operation latency, pure-Python reference (30 trials per measurement).

These numbers are dominated by the pure-Python ML-KEM implementation and are NOT representative of a production deployment. A constant-time C implementation of ML-KEM (such as the one bundled

with liboqs) runs each ML-KEM operation in tens of microseconds, 100– 1000× faster. In production the combiner (HKDF) and transcript hash add only single microseconds of overhead on top of the component KEMs.

3.5 Production-Hardening Gap

The reference implementation is suitable for academic exposition and KAT generation, but is NOT production-ready. The gap to production is substantial and is summarized below. Each item is addressed in the mitigation strategies of Section 7.

- **Constant-time enforcement.** Python’s bytecode interpreter makes constant-time guarantees impossible; the kyber-py library used here is also not constant-time. A production implementation must use a constant-time ML-KEM such as liboqs or pq-crystals/kyber, and the resulting binary must be verified using a tool like CT-Verif (Almeida-Barbosa-Barthe-Dupressoir, USENIX 2016), ctgrind, or Binsec/Rel (Daniel-Bardin-Rezk, EuroS&P 2020). See §6b and §7.
- **Masking for ML-KEM.** Higher-order ($d \geq 3$) Boolean+arithmetic masking of the NTT operations is required to defend against the differential power analyses of Pessl and Primas [28] and Ngo et al. [29].
- **Hardware-bound key storage.** Long-term sk_{ec} and sk_{lwe} must reside inside a FIPS 140-3 Level 3+ HSM, never in process memory. ECDH operations should be performed by the HSM’s certified curve engine.
- **Hardware true-random-number-generation seed.** Both KeyGen calls must draw randomness from an SP 800-90A approved DRBG seeded by a hardware TRNG. Python’s default os.urandom (and OpenSSL’s underlying RNG) are appropriate; an embedded deployment must use the platform’s certified RNG.
- **FrodoKEM for the LT tier.** The DHP-KEM-LT parameter set requires FrodoKEM-1344 alongside ML-KEM-1024. No maintained pure-Python FrodoKEM implementation exists; calling `keygen(DHP_KEM_LT)` in the reference therefore raises `NotImplementedError`.
- **Negotiation and key transport.** The reference does not implement the wire format (TLS-style negotiation, X.509 certificate carriers, etc.). Section 5 sketches the integration path.

4. Novelty

This section argues that DHP-KEM is non-trivially novel against the closest published schemes. The principal comparison is to the IETF X25519MLKEM768 hybrid (draft-ietf-tls-hybrid-design [13]), with secondary contrasts against ECIES [36, 38, 37] and the older classical “double-hard” schemes (e.g. Wei [35]).

4.1 Comparison with the IETF X25519MLKEM768 Hybrid

draft-ietf-tls-hybrid-design [13] is the closest existing scheme to DHP-KEM. At a high level, it concatenates the two ciphertexts and feeds the two shared secrets into the TLS 1.3 HKDF-based key schedule. DHP-KEM differs in three substantive ways:

20. Standalone security via explicit transcript binding. DHP-KEM feeds the transcript hash $T = H(A \parallel c_{lwe} \parallel pk_{ec} \parallel pk_{lwe})$ into HKDF-Extract as the salt parameter. The IETF X25519MLKEM768 hybrid achieves CCA security only when wrapped in the full TLS 1.3 handshake-message hashing schedule (Bindel et al. [12], §6); DHP-KEM therefore remains IND-CCA2 in standalone (non-TLS) deployments such as application-layer messaging, file-encryption-at-rest, and HSM-bound key wrap.
21. Defence in depth against structured-lattice cryptanalysis (research-tier LT mode). The DHP-KEM-LT parameter set layers FrodoKEM-1344 (unstructured LWE) alongside ML-KEM-1024 (structured Module-LWE). No published hybrid KEM combines these in a single combiner. The motivation is to hedge against an as-yet-unknown attack that specifically exploits the ring structure of Module-LWE — a possibility raised by ANSSI in its 2022 PQC guidance.
22. Explicit NCS-1:2020 parameter mapping with MODERATE and ADVANCED tiers. No prior hybrid KEM, to the authors’ knowledge, is specified end-to-end against the Kingdom’s NCS-1:2020 levels. Section 5 of this report gives the mapping in full, including residual gaps where NCS-1:2020 [19] predates FIPS 203 [10].

4.2 Comparison with ECIES

The Elliptic Curve Integrated Encryption Scheme (Bellare and Rogaway [36]; standardized in ISO 18033-2 [38] and ANSI X9.63 [37]) is the de-facto classical hybrid PKE. Like DHP-KEM, ECIES uses an ephemeral ECDH exchange and a KDF over the resulting shared secret. Unlike DHP-KEM, ECIES rests entirely on ECDLP, so any future CRQC running Shor’s algorithm renders every recorded ECIES ciphertext decryptable. This is not a theoretical risk: the harvest-now-decrypt-later threat model captured in NIST SP 1800-38B [42] explicitly assumes that ECDLP-based ciphertexts recorded today will be broken once a CRQC exists. DHP-KEM’s post-quantum component is the fundamental quantitative novelty over ECIES: even a successful Shor attack on the classical half leaves Module-LWE intact, so the session key K remains indistinguishable from random by Proposition 2. Beyond the post-quantum advantage, DHP-KEM additionally binds the transcript $T = H(A \parallel c_{lwe} \parallel pk_{ec} \parallel pk_{lwe})$ into HKDF-Extract’s salt parameter, which ECIES does not do; this gives DHP-KEM standalone IND-CCA2 guarantees outside the TLS layer.

4.3 Comparison with Stand-Alone ML-KEM

A natural alternative to DHP-KEM is to abandon the classical component entirely and deploy ML-KEM-768 (FIPS 203 [10]) on its own. This is the simpler path and is what NIST currently recommends for new applications post-2024. DHP-KEM’s case against pure ML-KEM rests on three considerations:

23. **Maturity-of-cryptanalysis asymmetry.** ECDLP has been the subject of nearly forty years of intensive cryptanalysis; the best-known classical attack (Pollard-rho) and quantum attack (Shor) have been understood for decades. Module-LWE was introduced in 2015 and its cryptanalytic understanding is comparatively young. The Chen 2024 episode [43], in which a claimed polynomial-time quantum algorithm for LWE was withdrawn within days, illustrates the kind of unsettled period that ECDLP passed through in the 1990s but Module-LWE is currently in. DHP-KEM’s hybrid design hedges against this asymmetry: if Module-LWE is broken before ECDLP’s quantum demise, the classical component still protects past sessions.
24. **Regulatory hedge.** Multiple national agencies — BSI Germany [26], ANSSI France, and the U.S. NSA Suite C NSA 2.0 — currently require or recommend hybrid (classical + PQ) constructions for cryptographic transitions, not pure post-quantum. The IETF draft-ietf-tls-hybrid-design [13] codifies this for TLS 1.3. Deploying pure ML-KEM today places organisations outside this hybrid-first regulatory consensus.
25. **Defence in depth against implementation flaws.** Implementations of mature primitives like X25519 have been extensively reviewed; ML-KEM implementations are newer and have already shown subtle implementation issues (e.g. the KyberSlash family [30]). A hybrid design ensures that an implementation flaw in one component does not lead to total session-key compromise.

These advantages come at a quantifiable cost: DHP-KEM-128 ciphertexts are $32 + 1088 = 1120$ bytes (vs ML-KEM-768’s 1088 bytes alone), and Encap is roughly $1.3\times$ slower than pure ML-KEM (Table 4). For most applications the bandwidth and latency overhead is negligible compared with the security margin gained; for resource-constrained deployments, the trade-off must be evaluated against the application’s confidentiality horizon.

4.4 Comparison with Classical “Double-Hard” Schemes

Several schemes from the 1990s and 2000s, including those by McCurley, Brickell and McCurley, and the digital-signature scheme of Wei [35], proposed combining ECDLP with the integer factorisation problem (IFP). These schemes provide no defence against a quantum adversary: both ECDLP and IFP succumb to Shor’s algorithm [14] in polynomial time on a CRQC. The class of two-hard-problem schemes is therefore meaningful only when at least one component is quantum-resistant. DHP-KEM is the first scheme, to the authors’ knowledge, to apply the two-hard-problem design philosophy specifically to the post-quantum migration context with a transcript-bound HKDF combiner. Recent post-quantum AKE work by Brendel et al. [34] explores complementary directions for messaging protocols such as Signal’s X3DH, but does not target the standalone-KEM setting addressed here.

4.5 Novelty Summary and Limitations

DHP-KEM's novelty rests on the conjunction of four properties, no proper subset of which is present in any existing scheme to the authors' knowledge:

- A disjunctive-secure HKDF Extract-then-Expand combiner with explicit transcript binding (§2.2.4), providing standalone IND-CCA2 security via Proposition 2 [11, 12].
- Standalone IND-CCA2 security outside of TLS's handshake-hashing context (§4.1) — a property that draft-ietf-tls-hybrid-design [13] explicitly does not provide.
- A research-tier LT parameter set that hedges against ring-structured-lattice cryptanalysis by adding FrodoKEM-1344 alongside ML-KEM-1024 (§4.1).
- The first end-to-end mapping of a hybrid KEM onto the Saudi NCS-1:2020 standard with explicit MODERATE and ADVANCED parameter sets (Section 5).

Honest limitations of this novelty claim: (i) the combiner construction itself is taken from Giacon-Heuer-Poettering [11] and Bindel et al. [12], not invented here — DHP-KEM's contribution is the specific transcript-binding choice and parameter mappings, not the combiner abstraction; (ii) the disjunctive dual-hardness property is shared with all hybrid KEMs, not unique to DHP-KEM; (iii) for purely TLS deployments, X25519MLKEM768 [13] is the more compatible choice precisely because it integrates into the TLS key-schedule rather than competing with it. DHP-KEM's niche is non-TLS deployments and NCS-1:2020-compliant national-infrastructure contexts.

5. Alignment with NCS

This section maps the DHP-KEM specification onto the Saudi National Cybersecurity Authority's cryptographic-standards stack. Five documents are relevant: NCS-1:2020 [19] (the central cryptographic standard, with MODERATE and ADVANCED levels), ECC-1:2018 [20] (Essential Cybersecurity Controls), CCC-1:2020 [21] (Cloud Cybersecurity Controls), CCCC-1:2019 [22] (Critical Systems Cybersecurity Controls), and DCC-1 [23] (Data Cybersecurity Controls). Table 5 below shows the NCA regulatory stack at a glance.

NCA document	Scope (relevant to DHP-KEM)
NCS-1:2020 [19]	Cryptographic-algorithm strength levels (MODERATE / ADVANCED); approved primitives and parameters; PQ-readiness clauses in Appendix B.
ECC-1:2018 [20] (Essential Cybersecurity Controls)	Baseline controls applicable to every government and critical-infrastructure entity, including cryptographic-controls clauses 2-2-2.
CCC-1:2020 [21] (Cloud Cybersecurity Controls)	Mandatory controls for cloud service providers and consumers; the cryptography sub-controls require NCS-1:2020-approved algorithms.
CCCC-1:2019 [22] (Critical Systems Cybersecurity Controls)	Additional controls layered on ECC-1 for critical systems; requires ADVANCED-level cryptography in many cases.
DCC-1 [23] (Data Cybersecurity Controls)	Data-classification regime (Public / Internal / Restricted / Secret / Top-Secret); each tier mandates a minimum cryptographic strength.

Table 5. Saudi NCA cybersecurity-standards stack.

5.1 Mapping to NCS-1:2020 — MODERATE Level

Table 6 maps the cryptographic requirements of the NCS-1:2020 MODERATE level onto the DHP-KEM-128 parameter set. Every requirement is satisfied (✓).

NCS-1:2020 element	MODERATE-level requirement	DHP-KEM-128 instantiation
Asymmetric KEM / KEX	ECDH on ≥ 256 -bit curve	ECDH on Curve25519 ✓
Hash function	SHA-256 or stronger	SHA-384 (transcript) ✓
KDF	HKDF or NIST SP 800-56C [39]	HKDF-SHA-384 ✓
Symmetric output strength	≥ 128 bits	128 bits ✓
Post-quantum readiness	Recommended (Appendix B)	ML-KEM-768 ✓ (NIST Cat. 3)
PRNG	NIST SP 800-90A approved	CTR_DRBG ✓ (OS-provided)
Side-channel resistance	Recommended (production-only)	Constant-time + masking required ✓ (§7)

Table 6. NCS-1:2020 MODERATE-level requirements vs DHP-KEM-128 instantiation.

5.2 Mapping to NCS-1:2020 — ADVANCED Level

The DHP-KEM-256 (ADVANCED) parameter set similarly satisfies every requirement of the ADVANCED level, which is mandated for Restricted and Secret data per DCC-1 [23]. The principal differences from the MODERATE level are: ECDH on ≥ 384 -bit curve, SHA-384 or stronger hash, ≥ 256 -bit symmetric output, mandatory PQ readiness, and mandatory side-channel resistance verified at FIPS 140-3 Level 3+. ML-KEM-1024 reaches NIST Cat. 5 [10], which corresponds to AES-256-equivalent classical security.

NCS-1:2020 element	ADVANCED-level requirement	DHP-KEM-256 instantiation
Asymmetric KEM / KEX	ECDH on ≥ 384 -bit curve	ECDH on NIST P-384 ✓
Hash function	SHA-384 or stronger	SHA3-512 (transcript) ✓
KDF	HKDF or NIST SP 800-56C [39]	HKDF-SHA-512 ✓
Symmetric output strength	≥ 256 bits	256 bits ✓
Post-quantum readiness	Required (Appendix B — mandatory in next revision)	ML-KEM-1024 ✓ (NIST Cat. 5)
PRNG	NIST SP 800-90A with HW seed	CTR_DRBG + HW TRNG ✓
Side-channel resistance	Mandatory — FIPS 140-3 Level 3+	Constant-time + masking + verified (§7) ✓

Table 7. NCS-1:2020 ADVANCED-level requirements vs DHP-KEM-256 instantiation.

5.3 Mapping to ECC-1, CCC-1, CSCC-1, and DCC-1

ECC-1:2018 [20] sub-control 2-2-2 requires that all cryptographic operations use NCS-1:2020-approved algorithms; DHP-KEM-128 and DHP-KEM-256 both satisfy this trivially by virtue of instantiating only NCS-1-listed primitives. CCC-1:2020 [21] inherits this requirement for cloud deployments; the additional cloud-specific consideration is key-wrapping at rest, for which DHP-KEM is suitable as a KEM-then-DEM construction. CSCC-1:2019 [22] requires the ADVANCED level for cryptographic operations in critical systems; the DHP-KEM-256 set is designed precisely for this case.

DCC-1 [23] classifies national data into four sensitivity levels. DHP-KEM-128 (MODERATE) is suitable for Public and Internal data; DHP-KEM-256 (ADVANCED) is required for Restricted and Secret data; DHP-KEM-LT (research-tier) is recommended for Top-Secret data with confidentiality horizons exceeding the projected arrival of fault-tolerant quantum computers.

5.4 Residual Gaps

NCS-1:2020 was published before FIPS 203 [10] was standardized and therefore does not list ML-KEM in its current edition. The Appendix-B PQ-readiness clauses anticipate this gap: PQ primitives are recommended at MODERATE and required at ADVANCED, pending an NCS-1 revision that explicitly enumerates ML-KEM-768 and ML-KEM-1024. Until that revision is issued, DHP-KEM deployments rely on this report’s mapping plus FIPS 203’s own status as the authoritative US-federal standard for the post-quantum component.

6. Security Considerations and Attacks

This section catalogues the attacks against DHP-KEM in three subsections: (a) classical cryptanalytic attacks against the underlying hard problems, (b) side-channel attacks against the implementation, and (c) quantum attacks. Each subsection in (a) and (b) is paired with a runnable demonstration in the accompanying code package under `code/attacks/`. Section 7 then pairs every identified attack with a concrete mitigation.

A note on verification: side-channel claims must be VERIFIED rather than asserted. Modern constant-time verifiers such as CT-Verif (Almeida-Barbosa-Barthe-Dupressoir, USENIX 2016), ctgrind, and Binsec/Rel (Daniel-Bardin-Rezk, EuroS&P 2020) can statically prove that a given binary executes in time and memory-access patterns independent of its secret inputs. Section 7 specifies these verifiers as required release-CI checks for any DHP-KEM implementation.

6.1 Cryptanalysis (6a)

Four classes of classical cryptanalytic attack are relevant to DHP-KEM: brute-force search on the symmetric output, Pollard’s rho on ECDLP, primal/dual lattice attacks on Module-LWE, and — looking ahead to §6c — the resource requirements for quantum attacks. Each is discussed below with reference to a runnable demonstration in `code/attacks/`.

6.1.1 Brute-Force Symmetric-Key Search

The DHP-KEM-128 session key K is 256 bits long but the effective symmetric strength is bounded by the smallest classical component: Curve25519’s $\approx 2^{126}$ bit-operations for ECDLP via Pollard’s rho. A brute-force search over a 128-bit key space is therefore the easier attack, and it is comprehensively infeasible. The demonstration in `code/attacks/brute_force.py` computes how long an exhaustive search would take at four illustrative hardware budgets:

```
# Excerpt – code/attacks/brute_force.py
def years_to_search(key_bits: int,
                    evals_per_second: float) -> float:
    """Return number of years to search
    2**(key_bits-1) keys on average at the given rate."""
    avg_searches = 2 ** (key_bits - 1)
    seconds = avg_searches / evals_per_second
    return seconds / (365.25 * 24 * 3600)
for log2_rate, label in [ (30, "1 GH/s (one CPU core)"),
                          (40, "1 TH/s (large GPU cluster)"),
                          (60, "1 EH/s (hypothetical exascale ASIC farm)"),
                          (80, "1 YH/s (all silicon ever ever produced)"), ]:
    yrs = years_to_search(128, 2 ** log2_rate)
    print(f" {label:<50s} {yrs:.2e} years")
```

Sample output (illustrative):

1 GH/s (one CPU core)	5.02e+21 years	1 TH/s (large GPU cluster)
4.90e+18 years	1 EH/s (hypothetical exascale ASIC)	4.68e+12 years
1 YH/s (all silicon ever produced)	4.46e+06 years	

Even at 2^{80} evaluations per second — vastly beyond anything achievable on Earth — the expected search time exceeds 4 million years. Brute-force key search against DHP-KEM-128 is infeasible under every imaginable hardware budget.

6.1.2 Pollard's Rho on ECDLP

Pollard's rho method [25] solves the discrete-log problem $Q = k \cdot G$ in $O(\sqrt{n})$ group operations, where n is the order of the prime-order subgroup. For Curve25519, $n \approx 2^{252}$, so the cost is $\approx 2^{126}$ elliptic-curve point additions — well beyond reach of any classical computer. The demonstration in `code/attacks/pollard_rho.py` implements the algorithm on a toy curve where it actually succeeds:

```
# Excerpt - code/attacks/pollard_rho.py # Toy curve: y^2 = x^3 + x + 14 over F_1009
# Base point G = (2, 348), prime order n = 1013. def pollard_rho_dl(Q, n, max_iters=200_000):
    """Solve Q = k.G for k mod n via Floyd cycle finding."""
    R1, a1, b1 = G, 1, 0 # tortoise
    R2, a2, b2 = _f(R1, a1, b1, Q, n) # hare starts one step ahead
    for step in range(max_iters):
        R1, a1, b1 = _f(R1, a1, b1, Q, n)
        R2, a2, b2 = _f(R2, a2, b2, Q, n)
        if R1 == R2: # Collision found: a1.G + b1.Q == a2.G + b2.Q
            num = (a2 - a1) % n
            den = (b1 - b2) % n
            if den == 0:
                return None
            return (num * pow(den, -1, n)) % n
    return None
```

Running the script with a random secret scalar (e.g. $k = 656$) yields:

```
Secret scalar k = 656, public Q = k.G = (388, 752) Running Pollard's rho ...
Recovered k = 656. Success. Scaling to Curve25519 (n approx 2^252): expected
sqrt(n) approx 2^126 iterations approx 10^38 group operations approx 10^25 years
at 10^13 operations/second (>> age of universe)
```

The same algorithm, applied to Curve25519, would take more than 10^{25} years even on exascale hardware. Pollard's rho is therefore the operative ECDLP bound for DHP-KEM-128, and it places the classical security floor at $\approx 2^{126}$.

6.1.3 Primal, Dual, and Hybrid Lattice Attacks on Module-LWE

The three best-known classical lattice attacks on LWE are the primal attack (embedding into a uSVP instance and solving with BKZ), the dual attack (finding a short vector in the dual lattice and using it as a distinguisher), and Howgrave-Graham's hybrid (lattice reduction combined with meet-in-the-middle). ML-KEM-768's parameters were chosen so that the cheapest of the three (typically the primal attack with BKZ-619 and best-known sieving) costs approximately 2^{181} classical bit-operations under the core-SVP heuristic (Albrecht, Player, and Scott [24]). The accompanying script `code/attacks/lwe_distinguisher.py` runs the dual attack on a tiny LWE instance where it succeeds ($n = 4, q = 31$):

```
# Excerpt - code/attacks/lwe_distinguisher.py def find_short_dual_vector(A, n, m, q, sigma_bound):
    """Brute-force search for v in {-1,0,1}^m such that v^T A = 0 mod q."""
    from itertools import combinations
    for support_size in (3, 4, 5, 6, 7, 8):
        for support in combinations(range(m), support_size):
            for signs in product((-1, 1), repeat=support_size):
                v = [0] * m
                for idx, s in zip(support, signs):
                    v[idx] = s
                row = [sum(v[i] * A[i][j] for i in range(m)) % q for j in range(n)]
                if all(r == 0 for r in row):
                    return v
    return None # Distinguisher: if b = A.s + e is a real LWE sample then # <v, b> = <v, e> is "small mod q". Otherwise (b uniform) it is # uniformly distributed.
```

Sample run on the toy instance:

```
Toy LWE: n=4, m=16, q=31, sigma=1 found v with Hamming weight 5: [-1, 0, -1, 0, 0, 0, 0, -1, 0, 0, 0, -1, 0, 0, 0, 0]
<v, b> centred = -2, bound = 5 ->
```

distinguisher says 'LWE' <v, uniform b> centred = -12, outside bound -> correctly rejected On real ML-KEM-768 (n=2304, q=3329) the analogous attack uses BKZ-619 with quantum-amplified sieving and costs approx 2^{181} classical bit-operations (LWE-Estimator [24], core-SVP heuristic).

The 2^{181} estimate substantially exceeds the $\kappa = 128$ symmetric target of DHP-KEM-128, providing comfortable security margin even against the most aggressive classical lattice cryptanalysis.

6.1.4 Cryptanalysis Summary

Under Proposition 2 (§2.2.7), DHP-KEM is IND-CCA2 secure as long as either ECDLP or Module-LWE remains hard. The classical-attack budgets are: $\approx 2^{126}$ (Pollard's rho on Curve25519) and $\approx 2^{181}$ (primal/dual/hybrid lattice attacks on ML-KEM-768). The dominant constraint is the weaker ECDH side; this is acceptable because DHP-KEM's design philosophy treats the classical component as a backstop against future PQC failures, not as the primary long-term defence. The full attack-vs-DHP-KEM cost summary is given in Table 8.

Attack class	Best-known cost (DHP-KEM-128)	Best-known cost (DHP-KEM-256)
Brute-force key search	$\approx 2^{128}$ classical	$\approx 2^{256}$ classical
Pollard-rho on ECDLP	$\approx 2^{126}$ (Curve25519)	$\approx 2^{192}$ (P-384)
Primal/dual/hybrid on Module-LWE	$\approx 2^{181}$ (ML-KEM-768, core-SVP)	$\approx 2^{256}$ (ML-KEM-1024)

Table 8. Best-known classical cryptanalytic attacks on DHP-KEM.

6.2 Side-Channel Attacks (6b)

Side-channel attacks exploit physical characteristics of an implementation — timing, power, electromagnetic emanation, cache state, or induced faults — rather than the abstract cryptographic primitive. Each of the four canonical classes below is paired with a runnable demonstration in `code/attacks/`. See Section 7 for the corresponding mitigations.

6.2.1 Timing Attacks

A timing attack succeeds when the wall-clock duration of a cryptographic operation depends on a secret input. Kocher's original 1996 attack [16] recovered RSA private keys by measuring decryption time variations across many ciphertexts. In the DHP-KEM context, the most consequential timing-leak surface is non-constant-time byte-string comparison — e.g. checking whether a decapsulated key matches an expected MAC tag, or whether ML-KEM's re-encryption check passes. The demonstration in `code/attacks/timing_attack.py` contrasts two implementations of byte-string equality:

```
# Excerpt - code/attacks/timing_attack.py
def naive_compare(a: bytes, b: bytes) -> bool:
    """INSECURE: exits at the first mismatching byte."""
    if len(a) != len(b):
        return False
    for x, y in zip(a, b):
        if x != y:
            return False
    return True
def constant_time_compare(a: bytes, b: bytes) -> bool:
    """SECURE: runs to completion regardless of where a, b differ."""
    if len(a) != len(b):
        return False
    accumulator = 0
    for x, y in zip(a, b):
        accumulator |= x ^ y
    return accumulator == 0
```

Timing the two functions over 4000 trials each with 32-byte tags:

```

naive_compare      (INSECURE)      median time, mismatch at byte 0   : 0.40 us
median time, mismatch at byte 31 : 0.98 us      ratio (late / early)      :
2.44      >>> LEAK detected: timing exposes mismatch position.
constant_time_compare (SECURE)      median time, mismatch at byte 0   : 1.46 us
median time, mismatch at byte 31 : 1.46 us      ratio                      :
1.00      -> No exploitable timing difference.

```

The naive version leaks the mismatch position with a 2.4× timing ratio; the constant-time version is statistically indistinguishable. This is the model that `hmac.compare_digest` in the Python standard library follows, and it is what the FIPS-203 reference implementation uses internally.

6.2.2 Simple Power Analysis (SPA) and Differential Power Analysis (DPA)

Power analysis attacks measure the power consumption of a cryptographic device to extract secret information. Simple Power Analysis (Kocher, Jaffe, and Jun [17]) recovers secrets from a single trace by visual inspection; Differential Power Analysis correlates many traces with hypothesised intermediate values. In the DHP-KEM context, SPA targets the scalar-multiplication of ECDH, and DPA targets the secret-coefficient-dependent NTT inside ML-KEM decapsulation (Pessl and Primas [28]; Ngo et al. [29]). Analogous DPA attacks have been demonstrated against other lattice-based primitives, including the FALCON signature scheme (Karabulut and Aysu [48]), reinforcing that masking countermeasures are essential across the lattice-cryptography family. The demonstration in `code/attacks/power_analysis.py` simulates SPA on a non-constant-time scalar multiplier:

```
# Excerpt - code/attacks/power_analysis.py def naive_double_and_add(k,
gen_trace=True):    """Each '1' bit triggers DBL AND ADD; each '0' bit only DBL.
Trace pattern leaks the bits of k."""    R = 0    trace = []    for bit in
bin(k)[2:]:        trace.append(("DBL", _noisy(COST_DOUBLE)))    # always double
R = R * 2        if bit == "1":            trace.append(("ADD", _noisy(COST_ADD)))
# only add if bit set        R = R + 1    return R, trace def
recover_bits_from_naive_trace(trace):    """Read off the scalar bits directly from
the DBL/ADD pattern."""    bits = []    i = 0    while i < len(trace):
assert trace[i][0] == "DBL"        if i + 1 < len(trace) and trace[i + 1][0] ==
"ADD":            bits.append("1"); i += 2        else:
bits.append("0"); i += 1    return "".join(bits)
```

Running with a 16-bit secret scalar $k = 53259$ (binary 1101000000001011):

```
NAIVE double-and-add:      trace length          = 22 ops (differs by k's Hamming
weight!)    recovered k from SPA = 53259 = 0b1101000000001011      match
= True     CONSTANT-TIME ladder:   trace length          = 32 ops (EXACTLY 2 *
n_bits, independent of k) operation sequence =
DADADADADADADADADADADADADADADA recovered k from SPA = (no exploitable
structure)
```

SPA recovers every bit of the secret scalar from the naive trace in $O(n_bits)$ operations. The Montgomery-ladder alternative emits an identical DBL/ADD sequence for every key, leaking no information through op-counts or sequence patterns.

6.2.3 Fault-Injection Attacks

Fault-injection attacks (Boneh, DeMillo, and Lipton [18]) deliberately corrupt intermediate values during a cryptographic computation, then exploit the resulting faulty output to recover secrets. Modern delivery

mechanisms include voltage glitching, laser injection, clock-jitter induction, and Rowhammer. In the DHP-KEM context, the most relevant target is Kyber's decapsulation: Hermelink, Pessl, and Pöppelmann [31] showed that single-bit faults can extract sk_{lwe} . The demonstration in `code/attacks/fault_injection.py` runs the Boneh-DeMillo-Lipton attack on toy-scale ECDH:

```
# Excerpt - code/attacks/fault_injection.py # Toy curve: y^2 = x^3 + x + 14 over F_1009
# 1. Attacker observes honest ss = sk . A. # 2. Attacker triggers a fault during a second computation, # flipping bit 'bp' of sk. The faulty value is #
sk_faulty = sk XOR (1 << bp) # 3. Faulty result ss_faulty = sk_faulty . A leaks an #
algebraic relation involving sk. sk = 465 # Bob's real secret
sk_faulty = sk XOR (1 << 3) # attacker flips bit 3 ss_honest = scalar_mult(sk, A)
ss_faulty = scalar_mult(sk_faulty, A) # attacker learns ss_honest and ss_faulty via the protocol's # downstream MAC oracle. By trying all single-bit flips, # they can solve for sk algebraically.
```

Defences include redundant computation (compute the scalar multiplication twice and compare; abort on mismatch) and the explicit-rejection FO variant for Kyber's decapsulation, both discussed in Section 7.

6.2.4 Cache and Microarchitectural Attacks

Modern CPUs share caches, branch predictors, and TLBs across processes, creating a rich side-channel surface. Flush+Reload (Yarom and Falkner [32]), Prime+Probe (Liu et al.), Spectre/Meltdown (Kocher et al.), and the more recent PortSmash (Aldaya et al. [47]) and Hertzbleed (Wang et al. [33]) attacks have all been weaponized against cryptographic implementations. The demonstration in `code/attacks/cache_attack.py` simulates Flush+Reload against two S-box-lookup implementations:

```
# Excerpt - code/attacks/cache_attack.py def naive_lookup(idx): """Direct memory access at secret-dependent index. Leaks idx via cache state.""" return SBOX[idx]
def masked_lookup(idx): """Read every entry; arithmetic-mask out the right one. Constant-cache-pattern regardless of idx.""" result = 0
for i, entry in enumerate(SBOX): # mask is all-1s if i == idx, else all-0s
mask = -int(i == idx) & 0xFF result |= entry & mask return result
```

After a victim call with secret `idx = 137`, the simulated attacker measures reload time for every cache line and finds line 137 fast (cached) while all others are slow. Against `masked_lookup`, every line was touched, so reload times are uniform and `idx` is not recoverable.

6.3 Quantum Attacks (6c)

Quantum attacks against DHP-KEM split into three categories: Shor's algorithm against ECDLP, Grover's algorithm against the symmetric output, and quantum-amplified lattice sieving against Module-LWE.

6.3.1 Shor on ECDLP

Shor's algorithm [14] solves ECDLP in polynomial quantum time, so any sufficient CRQC breaks the classical half of DHP-KEM. The accompanying script `code/attacks/shor_estimate.py` applies the Roetteler-Naehrig-Svore-Lauter [45] and Häner-Roetteler-Soeken [27] resource estimates:

```
# Excerpt - code/attacks/shor_estimate.py def shor_ecdlp_logical(n_bits):
"""Logical-qubit / T-gate estimates per Roetteler-Naehrig-Svore-Lauter [45].""" q
```

```
= 9 * n_bits + 2 * math.ceil(math.log2(n_bits)) + 10      t = 448 * (n_bits ** 3) *
math.log2(n_bits)      return {"logical_qubits": q, "T_gates": t}
```

Sample output:

```
X25519 (DHP-KEM-128): ~2330 logical qubits, ~10^9 T-gates,
~tens of millions of physical qubits all-in. P-384 (DHP-KEM-256): ~3470 logical
qubits, ~10^10 T-gates. RSA-2048 (comparison): ~6160 logical qubits, ~10^12 T-
gates.
```

6.3.2 Grover on the Symmetric Output

Grover’s algorithm [15] halves the effective bit-strength of symmetric primitives. For DHP-KEM-128’s 256-bit output, Grover reduces the effective quantum-search-resistance to $\approx 2^{128}$ — still cryptographically strong, and unchanged from classical bounds for the purposes of $\kappa = 128$ alignment. The output length $L = 32$ bytes is therefore retained.

6.3.3 Quantum-Amplified Lattice Sieving and the Chen 2024 Episode

The security of Module-LWE under quantum adversaries is more subtle. Quantum sieving algorithms (Laarhoven [44]; Albrecht et al. [46]) reduce the cost of SVP from $2^{0.292\beta}$ classically to approximately $2^{0.265\beta}$ quantumly, providing a modest speedup. In April 2024, Chen [43] posted a manuscript claiming a polynomial-time quantum algorithm for LWE under specific parameter regimes; the manuscript was rapidly revised after Hongxun Wu identified a flaw in step 9 of the algorithm, and the corrected version does not break Kyber-class parameters. The episode nonetheless stimulated renewed cryptanalytic scrutiny of Module-LWE. Under the most aggressive quantum cost models currently accepted, ML-KEM-768 retains approximately 2^{164} bits of effective quantum security (core-SVP, quantum sieving).

It bears emphasising that no cryptographically-relevant quantum computer currently exists at the scale required by these resource estimates: fault-tolerant overheads are on the order of tens of millions of physical qubits, far beyond present hardware (Häner et al. [27]). The post-quantum threat addressed by DHP-KEM is therefore one of harvest-now-decrypt-later: recorded ciphertexts protected only by ECDH today could be decrypted at some future date when a CRQC arrives. This framing matches the threat horizon described in NCS-1:2020 [19] Appendix B.

7. Mitigation

This section pairs every attack identified in Section 6 with a concrete mitigation. The pairing is one-for-one and is summarized in Table 9; each row is then discussed in detail in the subsections below.

Attack class (§6)	Mitigation strategy
6.1.1 Brute-force key search	Use parameter sets DHP-KEM-128 ($\kappa = 128$) or DHP-KEM-256 ($\kappa = 256$) per the NCS-1:2020 mapping in Section 5. Brute-force is infeasible at every realistic hardware budget.
6.1.2 Pollard-rho on ECDLP	Curve choice: Curve25519 (DHP-KEM-128) or NIST P-384 (DHP-KEM-256). Both place rho beyond reach. Eventually migrate to a fully-PQ KEM as Module-LWE alone matures and gains additional cryptanalytic confidence.
6.1.3 Primal/dual/hybrid lattice	Parameter selection via the public LWE-Estimator [24] under the core-SVP heuristic. For defence in depth, deploy the DHP-KEM-LT (research-tier) parameter set adding FrodoKEM-1344, which protects against potential structured-lattice cryptanalysis advances.
6.2.1 Timing attacks	Constant-time implementations of every secret-dependent operation; use <code>hmac.compare_digest</code> or equivalent for tag comparison; verify the compiled binary with CT-Verif, ctgrind, or Binsec/Rel as a release-CI gate.
6.2.2 SPA / DPA	Montgomery-ladder scalar multiplication for the ECDH component; higher-order ($d \geq 3$) Boolean+arithmetic masking of the NTT inside ML-KEM decapsulation per Pessl-Primas [28] and Ngo et al. [29]; physical countermeasures (shielding, decoupling capacitors, clock-jitter) for the hardware device.
6.2.3 Fault injection	Redundant computation (run scalar multiplication twice; abort on mismatch); the explicit-rejection FO variant for ML-KEM per Hermelink-Pessl-Pöppelmann [31]; physical hardening of the device.
6.2.4 Cache / microarchitectural	Avoid all secret-dependent memory addresses; bit-sliced or vectorised constant-time S-box implementations; hardware cache-partitioning (Intel CAT) for high-assurance deployments; mitigation of Spectre v1 via speculative-execution barriers per platform guidance.
6.3.1 Shor on ECDLP	The post-quantum half of DHP-KEM (ML-KEM-768 or ML-KEM-1024) provides protection: even an unlimited CRQC attacker who breaks ECDH cannot derive K because they still face Module-LWE. This is the core design property (Proposition 2).
6.3.2 Grover on output	The 256-bit (32-byte) output K reduces under Grover to 2^{128} effective quantum work, meeting NIST Cat 1's quantum-security floor and matching the $\kappa = 128$ claim of DHP-KEM-128. For data with confidentiality horizons beyond projected CRQC arrival, deploy DHP-KEM-256 (output 256 bits $\rightarrow 2^{128}$ quantum work against a $\kappa = 256$ target, NIST Cat 5).
6.3.3 Quantum lattice cryptanalysis	ML-KEM-768 retains $\approx 2^{164}$ bits of quantum security under core-SVP heuristic. For Top-Secret data, DHP-KEM-LT layers FrodoKEM-1344 alongside ML-KEM to hedge against future ring-structured-lattice breakthroughs.

Table 9. Attack-to-mitigation pairing for DHP-KEM.

7.1 Cryptanalytic Mitigations

DHP-KEM's parameter sets are chosen so that the cheapest classical attack against either component costs at least 2^{126} bit-operations (the Curve25519 ECDLP floor for DHP-KEM-128) or 2^{181} (the ML-KEM-768 lattice floor under core-SVP). Both substantially exceed the $\kappa = 128$ symmetric target. The mitigation strategy here is not active defence but principled parameter selection: rely on the public LWE-Estimator [24] under the most conservative cost model accepted by NIST during the FIPS 203 [10] standardization, and track refinements as the cryptanalytic literature evolves.

For deployments seeking margin against potential future cryptanalytic advances, the DHP-KEM-LT (research-tier) parameter set additionally layers FrodoKEM-1344 (unstructured LWE) alongside ML-KEM-1024 (structured Module-LWE). An attacker would then need to break BOTH structured-lattice Module-LWE AND unstructured-lattice LWE — a much more demanding requirement. This is the standard “defence in depth” argument: even if a future ring-structured-lattice cryptanalysis significantly reduces Module-LWE's security, the FrodoKEM half remains intact because FrodoKEM does not use any ring structure. Feasibility: DHP-KEM-LT roughly doubles ciphertext size and computational cost compared with DHP-KEM-256; this is acceptable for low-volume high-assurance use cases (Top-Secret diplomatic communications, root-CA key transport) but probably impractical for general TLS handshake deployment.

Pollard-rho on the ECDLP component is unmitigated by parameter changes within the X25519 or P-384 families; the curve choice already places rho beyond reach (2^{126} and 2^{192} respectively). The longer-term mitigation against ECDLP — anticipating a CRQC — is the post-quantum component itself: by Proposition 2, breaking ECDLP via Shor leaves the session key K still indistinguishable from random as long as Module-LWE is hard. This is the principal design property of DHP-KEM.

7.2 Side-Channel Mitigations

Side-channel claims must be verified, not asserted. Three distinct categories of countermeasure are required, addressing different leak surfaces. Each is summarized below with its feasibility profile.

Algorithmic countermeasures (constant-time code). Every secret-dependent operation must run in constant time: no branches, no memory accesses, no arithmetic operations whose execution path or duration depends on secret data. For ECDH, this is achieved by the Montgomery-ladder scalar multiplication used by X25519 (Bernstein [4]). For ML-KEM, the FIPS 203 reference implementation includes constant-time hooks but actual constant-time behaviour must be re-verified per compiler and per target ISA (the KyberSlash episode [30] showed that compiler-introduced division timings can re-introduce leakage even when the source code is constant-time). Feasibility: constant-time implementations of both components are freely available (libsodium, liboqs, pq-crystals), so the practical cost is essentially zero — only the verification work.

Algorithmic countermeasures (masking). Higher-order ($d \geq 3$) Boolean+arithmetic masking of the secret-dependent NTT inside ML-KEM decapsulation is required to defend against the chosen-ciphertext DPA attacks of Pessl and Primas [28] and Ngo et al. [29]. A d -th order masking scheme increases the number

of power traces required for a successful DPA by approximately 2^d . Feasibility: masking implementations are available in research code (pq-crystals/kyber-masked) but require careful integration and have a 5–10× performance overhead. For an HSM deployment this overhead is absorbed; for a TLS handshake it may be marginal but is currently the only known defence against laboratory-grade DPA.

Verification. Even a correctly-written constant-time algorithm may be compiled to leaky machine code. The compiled binary must therefore be verified by at least one of CT-Verif (Almeida et al., USENIX 2016), ctgrind (Langley, 2010), or Binsec/Rel (Daniel-Bardin-Rezk, EuroS&P 2020), against the actual secret-key types of the implementation. These verifiers produce a positive certificate or a counter-example trace; we recommend including the certificate as a release-CI artifact. Feasibility: all three verifiers are open-source and supported for x86-64; CT-Verif requires LLVM bitcode (compatible with any LLVM toolchain), ctgrind runs at Valgrind speeds (~10× slowdown but only during CI runs), and Binsec/Rel operates on stripped binaries. Verifier integration is a one-day engineering task for an existing CI pipeline.

Physical countermeasures. Hardware-resident DHP-KEM implementations (smart cards, HSMs, secure-element ICs) must additionally include shielding to attenuate electromagnetic emanation [17], decoupling capacitors to flatten supply-current variations, clock-jitter injection to defeat trace alignment, redundant scalar-multiplication checks to detect single-bit fault injection [18, 31], and tamper-evident enclosures for physical access deterrence. Feasibility: all of these are standard practice for Common Criteria EAL 5+ and FIPS 140-3 Level 3+ devices, and certified components are commercially available.

7.3 Implementation-Hygiene Mitigations

Beyond the cryptanalytic and side-channel categories above, several operational hygiene items are required for a production DHP-KEM deployment:

- Long-term sk_{ec} and sk_{lwe} reside inside a FIPS 140-3 Level 3+ HSM, never in process memory. This neutralises in-memory key extraction attacks (cold-boot, process memory dumps, Heartbleed-class bugs).
- Both KeyGen calls draw randomness from a NIST SP 800-90A approved DRBG seeded by a hardware TRNG. Predictable randomness has historically been a more common vulnerability than algorithmic weakness (Debian OpenSSL 2006-2008; Sony PlayStation ECDSA 2010); this is non-negotiable.
- Keys are rotated according to NCS-1:2020 lifecycle rules: ADVANCED tier requires rotation no less frequent than every 3 years for long-term identities, with overlap windows for graceful transition. This bounds the blast radius of any future key compromise.
- Negotiation of parameter sets between Alice and Bob is performed through a signed announcement (not in-band), preventing downgrade attacks on the parameter-set choice itself. This is the same defence pattern as TLS 1.3's NamedGroup signature inclusion.
- Composite-key carriers in X.509 certificates follow the IETF composite-keys draft [41]. This standardizes the wire format and reduces interoperability risk.

All seven items above (three in §7.2 plus four implementation-hygiene items in §7.3) should be enforced as gating release-CI checks. An implementation that lacks any one of them is not yet production-ready, regardless of how clean its cryptographic core is.

8. Conclusion

This report has specified DHP-KEM, a hybrid key-encapsulation mechanism that combines ECDH on Curve25519 (or NIST P-384) with ML-KEM-768 (or ML-KEM-1024) through a transcript-bound HKDF Extract-then-Expand combiner. The encapsulated session key K remains pseudorandom as long as at least one of the two underlying hard problems — ECDLP or Module-LWE — remains computationally intractable, captured by the disjunctive dual-hardness statement “ECDLP-hard $\vee$ Module-LWE-hard” (Proposition 2).

The protocol’s design is documented in Section 2, including the underlying mathematical equations (§2.2), the step-by-step procedures for Alice and Bob (§2.3), and a complete worked example with real Curve25519 + ML-KEM-768 hex bytes (§2.4) that is independently reproducible from the accompanying code package. Section 3 documented the reference implementation, its five-module library design, the 23-test test suite (all passing), and the production-hardening gap. Section 4 argued novelty against the closest published scheme, the IETF X25519MLKEM768 hybrid, by reference to three substantive distinguishing features. Section 5 mapped DHP-KEM end-to-end onto the Saudi NCA NCS-1:2020 standard and adjacent regulatory documents.

Section 6 catalogued cryptanalytic, side-channel, and quantum attacks against the protocol, with eight runnable attack demonstrations in the accompanying code package illustrating each attack class. Section 7 paired every identified attack with a concrete mitigation. Three parameter sets—MODERATE, ADVANCED, and LT (research-tier)—were introduced and aligned with the NCS-1:2020 levels.

Future work falls into three areas. First, implementing the DHP-KEM-LT FrodoKEM-1344 second post-quantum component, which the present reference omits because no maintained pure-Python FrodoKEM library is available; a C-based implementation via liboqs would close this gap. Second, formal verification of the combiner: while Proposition 2 is restated from the peer-reviewed Giacon-Heuer-Poettering [11] / Bindel et al. [12] arguments, a mechanised EasyCrypt or Tamarin proof would strengthen the assurance argument. Third, integration testing against the Saudi national PKI infrastructure (Absher and IAM identity infrastructure), including end-to-end interoperability and timing studies on certified FIPS 140-3 hardware.

9. References

- [1] W. Diffie and M. E. Hellman, “New directions in cryptography,” *IEEE Trans. Inf. Theory*, vol. 22, no. 6, pp. 644–654, Nov. 1976.
- [2] V. S. Miller, “Use of elliptic curves in cryptography,” in *Advances in Cryptology – CRYPTO ’85*, LNCS 218, Springer, 1986, pp. 417–426.
- [3] N. Koblitz, “Elliptic curve cryptosystems,” *Mathematics of Computation*, vol. 48, no. 177, pp. 203–209, 1987.
- [4] D. J. Bernstein, “Curve25519: New Diffie–Hellman speed records,” in *Public Key Cryptography – PKC 2006*, LNCS 3958, Springer, 2006, pp. 207–228.
- [5] NIST Special Publication 800-186, *Recommendations for Discrete Logarithm-based Cryptography: Elliptic Curve Domain Parameters*, Feb. 2023.
- [6] O. Regev, “On lattices, learning with errors, random linear codes, and cryptography,” *Journal of the ACM*, vol. 56, no. 6, art. 34, 2009.
- [7] V. Lyubashevsky, C. Peikert, and O. Regev, “On ideal lattices and learning with errors over rings,” *Journal of the ACM*, vol. 60, no. 6, art. 43, 2013.
- [8] A. Langlois and D. Stehlé, “Worst-case to average-case reductions for module lattices,” *Designs, Codes and Cryptography*, vol. 75, no. 3, pp. 565–599, 2015.
- [9] J. Bos et al., “CRYSTALS–Kyber: A CCA-secure module-lattice-based KEM,” in *Proc. IEEE European Symposium on Security and Privacy (EuroS&P)*, 2018, pp. 353–367.
- [10] NIST, *FIPS PUB 203, Module-Lattice-Based Key-Encapsulation Mechanism Standard*, Aug. 2024.
- [11] F. Giacon, F. Heuer, and B. Poettering, “KEM combiners,” in *Public Key Cryptography – PKC 2018*, LNCS 10769, Springer, 2018, pp. 190–218.
- [12] N. Bindel, J. Brendel, M. Fischlin, B. Goncalves, and D. Stebila, “Hybrid key encapsulation mechanisms and authenticated key exchange,” in *Post-Quantum Cryptography – PQCrypto 2019*, LNCS 11505, Springer, 2019, pp. 206–226.
- [13] D. Stebila, S. Fluhrer, and S. Gueron, “Hybrid key exchange in TLS 1.3,” *IETF draft, draft-ietf-tls-hybrid-design*, 2024.
- [14] P. W. Shor, “Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer,” *SIAM J. Computing*, vol. 26, no. 5, pp. 1484–1509, 1997.
- [15] L. K. Grover, “A fast quantum mechanical algorithm for database search,” in *Proc. 28th ACM STOC*, 1996, pp. 212–219.
- [16] P. C. Kocher, “Timing attacks on implementations of Diffie–Hellman, RSA, DSS, and other systems,” in *Advances in Cryptology – CRYPTO ’96*, LNCS 1109, Springer, 1996, pp. 104–113.

- [17] P. Kocher, J. Jaffe, and B. Jun, “Differential power analysis,” in *Advances in Cryptology – CRYPTO ’99*, LNCS 1666, Springer, 1999, pp. 388–397.
- [18] D. Boneh, R. A. DeMillo, and R. J. Lipton, “On the importance of checking cryptographic protocols for faults,” in *Advances in Cryptology – EUROCRYPT ’97*, LNCS 1233, Springer, 1997, pp. 37–51.
- [19] Saudi National Cybersecurity Authority, NCS-1:2020 — National Cryptographic Standards, 2020.
- [20] Saudi National Cybersecurity Authority, ECC-1:2018 — Essential Cybersecurity Controls, 2018.
- [21] Saudi National Cybersecurity Authority, CCC-1:2020 — Cloud Cybersecurity Controls, 2020.
- [22] Saudi National Cybersecurity Authority, CSCC-1:2019 — Critical Systems Cybersecurity Controls, 2019.
- [23] Saudi National Cybersecurity Authority, DCC-1 — Data Cybersecurity Controls, 2022.
- [24] M. R. Albrecht, R. Player, and S. Scott, “On the concrete hardness of learning with errors,” *Journal of Mathematical Cryptology*, vol. 9, no. 3, pp. 169–203, 2015.
- [25] J. M. Pollard, “Monte Carlo methods for index computation (mod p),” *Mathematics of Computation*, vol. 32, no. 143, pp. 918–924, 1978.
- [26] BSI (Germany), Technical Guideline TR-02102-1 — Cryptographic Mechanisms: Recommendations and Key Lengths, 2024.
- [27] T. Häner, M. Roetteler, and M. Soeken, “Improved quantum circuits for elliptic curve discrete logarithms,” in *Post-Quantum Cryptography – PQCrypto 2020*, LNCS 12100, Springer, 2020, pp. 425–444.
- [28] P. Pessl and R. Primas, “More practical single-trace attacks on the number theoretic transform,” in *LATINCRYPT 2019*, LNCS 11774, Springer, 2019, pp. 130–149.
- [29] K. Ngo, E. Dubrova, Q. Guo, and T. Johansson, “A side-channel attack on a masked IND-CCA secure SABER KEM implementation,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2021, no. 4, pp. 676–707, 2021.
- [30] D. J. Bernstein, K. Bhargavan, S. Cheval, et al., “KyberSlash: Exploiting secret-dependent division timings in Kyber implementations,” *IACR ePrint 2024/1049*, 2024.
- [31] J. Hermelink, P. Pessl, and T. Pöppelmann, “Fault-enabled chosen-ciphertext attacks on Kyber,” in *INDOCRYPT 2021*, LNCS 13143, Springer, 2021, pp. 311–334.
- [32] Y. Yarom and K. Falkner, “Flush+Reload: A high-resolution, low-noise, L3 cache side-channel attack,” in *Proc. 23rd USENIX Security Symposium*, 2014, pp. 719–732.
- [33] Y. Wang et al., “Hertzbleed: Turning power side-channel attacks into remote timing attacks on x86,” in *Proc. 31st USENIX Security Symposium*, 2022, pp. 679–697.
- [34] J. Brendel, M. Fischlin, F. Günther, C. Janson, and D. Stebila, “Towards post-quantum security for Signal’s X3DH handshake,” in *Selected Areas in Cryptography – SAC 2022*, LNCS 13742, Springer, 2022, pp. 404–430.

- [35] C. H. Wei, “A digital signature scheme based on two hard problems,” *Int. Journal of Network Security*, vol. 5, no. 3, pp. 263–267, Nov. 2007.
- [36] M. Bellare and P. Rogaway, “Minimizing the use of random oracles in authenticated encryption schemes,” in *Information and Communications Security (ICICS ’97)*, LNCS 1334, Springer, 1997, pp. 1–16.
- [37] ANSI X9.63-2017, *Public Key Cryptography for the Financial Services Industry — Key Agreement and Key Transport Using Elliptic Curve Cryptography*, 2017.
- [38] ISO/IEC 18033-2:2006, *Information Technology — Security Techniques — Encryption Algorithms — Part 2: Asymmetric Ciphers*.
- [39] NIST Special Publication 800-56C Rev. 2, *Recommendation for Key-Derivation Methods in Key-Establishment Schemes*, Aug. 2020.
- [40] H. Krawczyk and P. Eronen, “HMAC-based Extract-and-Expand Key Derivation Function (HKDF),” RFC 5869, May 2010.
- [41] M. Ounsworth and J. Gray, “Composite ML-KEM for use in the Internet X.509 Public Key Infrastructure,” IETF draft, draft-ietf-lamps-pq-composite-keys, 2024.
- [42] NIST National Cybersecurity Center of Excellence, “Migration to Post-Quantum Cryptography,” NIST SP 1800-38B, Apr. 2024.
- [43] Y. Chen, “Quantum Algorithms for Lattice Problems,” IACR ePrint Archive Report 2024/555, Apr. 2024 (revised May 2024 following identification of a flaw by H. Wu).
- [44] T. Laarhoven, “Search problems in cryptography: From fingerprinting to lattice sieving,” Ph.D. dissertation, Eindhoven University of Technology, 2015.
- [45] M. Roetteler, M. Naehrig, K. M. Svore, and K. Lauter, “Quantum resource estimates for computing elliptic curve discrete logarithms,” in *Advances in Cryptology – ASIACRYPT 2017*, LNCS 10625, Springer, 2017, pp. 241–270.
- [46] M. R. Albrecht, L. Ducas, G. Herold, E. Kirshanova, E. W. Postlethwaite, and M. Stevens, “The general sieve kernel and new records in lattice reduction,” in *Advances in Cryptology – EUROCRYPT 2019*, LNCS 11477, Springer, 2019, pp. 717–746.
- [47] A. C. Aldaya, B. B. Brumley, S. ul Hassan, C. P. García, and N. Tuveri, “Port contention for fun and profit,” in *Proc. IEEE Symposium on Security and Privacy*, 2019, pp. 870–887.
- [48] E. Karabulut and A. Aysu, “FALCON down: Breaking FALCON post-quantum signature scheme through side-channel attacks,” in *Proc. 58th ACM/IEEE Design Automation Conference (DAC)*, 2021, pp. 691–696.